

- Drill
- Glossary
- Cheat Sheet
- Curriculum
- Mistake Report

NCP-AAI Repeat-Mistake Drill

All topics

- CertyIQ Repeat Offenders (24, MCQ)
- Preporato Review (44, MCQ)
- Session History

24 questions in this filter Score this session: 17/24

Session complete — CertyIQ Repeat Offenders

71%

17 of 24 correct

By topic (weakest first)

Knowledge Integration & Data Handling		0/3
Run, Monitor & Maintain		2/3
Deployment & Scaling		4/6
Evaluation & Tuning		2/3
Safety, Ethics & Compliance		3/3
Human-AI Interaction & Oversight		3/3
Cognition, Planning & Memory		3/3

What to review (7 missed this session)

- You're building a RAG system that uses RAG Fusion. Which of the following approaches would be most effective in determining how to combine information from multiple retrieved chunks?

You answered: **Concatenating the text from all retrieved chunks into a single block to form the response.**

Correct: **Using the LLM to automatically identify the most important sentences within each chunk and combine them.**

RAG Fusion relies on the LLM itself to identify and combine the most salient content across chunks rather than naive concatenation.
- An autonomous vehicle company operates a multi-agent AI system across its fleet to process real-time sensor data, make driving decisions, and communicate with cloud infrastructure. The company needs fleet-wide monitoring to track GPU utilization, inference times, and memory usage, correlate

performance with driving conditions and system load, and predict safety issues before they occur. Which monitoring and observability approach would BEST meet these fleet-scale, safety-critical requirements?

You answered: **Deploy enterprise telemetry using OpenTelemetry standards with machine learning-based anomaly detection, custom performance visualization, and automated alerting to deliver predictive operational insights and support proactive maintenance actions.**

Correct: **Deploy NVIDIA NIM microservices with Prometheus integration, NVIDIA Nsight Systems profiling, and Kubernetes-native monitoring to provide detailed metrics, profiling, and container orchestration observability across the entire stack.**

When one option names the full NVIDIA-native stack (NIM + Prometheus + Nsight Systems + Kubernetes) built for exactly the described need, it is almost always the intended answer over a generic APM/OpenTelemetry option.

Your agent is designed to manage tasks through a service management API. The API responds with detailed event logs, but these logs contain both metadata and structured data. To ensure the agent correctly interprets and processes the data from these logs, what's the most prudent approach?

You answered: **Employing a modular design that allows the agent to dynamically adjust its parsing logic.**

Correct: **Employ a specialized parser that adheres to the API's documentation, to insure strict adherence to structured data.**

A documentation-conformant, schema-aware parser is the reliable answer — 'dynamically adjust parsing logic' sounds flexible but is actually a reliability risk (undocumented/ad-hoc behavior).

When evaluating a multi-agent customer service system experiencing unpredictable scaling costs and performance bottlenecks during peak hours, which analysis approaches effectively identify optimization opportunities for both infrastructure efficiency and service reliability? Pick the 2 correct responses below

You answered: **Deploy distributed tracing with cost attribution per agent type, correlating resource consumption with business value metrics to identify optimization opportunities in agent deployment strategies. + Deploy agents with configurable scaling workflows, allowing analysis of resource adjustment strategies and their effects on service stability during variable demand periods.**

Correct: **Deploy agents with configurable scaling workflows, allowing analysis of resource adjustment strategies and their effects on service stability during variable demand periods. + Implement comprehensive workload profiling using NVIDIA Nsight to analyze GPU utilization patterns, identify underutilized resources, and optimize batch sizing for dynamic scaling with Kubernetes HPA.**

Configurable scaling analysis plus concrete NVIDIA-stack profiling/HPA tuning is the intended pair — cost-attribution tracing sounds relevant but is the distractor here.

You are designing the architecture for a RAG (Retrieval-Augmented Generation) system, and you are concerned about ensuring data freshness and minimizing latency. Which of the following is the most important consideration when designing the architecture?

You answered: **Implementing a single, centralized database for all data, updated with a synchronous polling mechanism for the LLM to retrieve the latest information.**

Correct: **Use a loosely coupled, event-driven micro-service architecture where separate services handle data indexing, retrieval, and LLM prompting.**

Event-driven, decoupled microservices scale and stay fresh without synchronous polling bottlenecks — any option built around a single centralized/synchronous store is the trap.

Which two validation approaches are MOST critical for ensuring agent reliability in production deployments? (Choose two.)

You answered: **Structured output validation with Pydantic schemas**

Correct: **Structured output validation with Pydantic schemas; Automated consistency checking across multiple agent runs**

Schema-based output validation and automated cross-run consistency checks are objective and continuous — user satisfaction surveys are subjective/lagging, not a reliability validation mechanism.

When analyzing throughput bottlenecks in a multi-modal agent processing text, images, and audio, which Triton configuration evaluations identify optimization opportunities? (Choose two.)

You answered: **Profile GPU memory allocation patterns across modalities, implement model instance batching strategies, and tune concurrency limits to maximize utilization. + Deploy each modality on separate Triton instances, allowing Triton to automatically manage ensemble coordination, shared memory usage, and pipeline integration.**

Correct: **Analyze model ensemble pipelines for sequential dependencies, identify parallelization opportunities, and optimize inter-model data transfer using Triton's scheduler. + Profile GPU memory allocation patterns across modalities, implement model instance batching strategies, and tune concurrency limits to maximize utilization.**

Optimization requires deliberate profiling and tuning — options that describe Triton 'automatically' handling coordination/concurrency without your own profiling are the traps.

[Replay 7 missed only →](#)

[Restart this filter](#)

Local-only, no server. Progress and session history are stored in this browser's localStorage (ncp_drill_progress, ncp_drill_sessions) — nothing leaves your machine.